

REACT.JS

DAY 1 - Getting Started with React

→ React, a short Introduction

- A popular JavaScript library for building user interfaces, particularly for single-page applications.
 - **Single Page Application (SPA)** - entire application is contained within a single HTML page.
 - Loads a single HTML page and dynamically updates the content without requiring a full page reload.
 - It provides a faster response because there's no waiting to reload entire pages.
 - Only the required data is fetched and updated.
 - In SPAs, routing is handled on the client side using libraries like React Router, so different views or pages are displayed without refreshing the whole page.
- Can create reusable UI components, making it easier to manage and update.
- Developed by Facebook and released in 2013.
- Not a framework.
- ❖ **create-react-app**
 - It supports a development environment with minimal setup.
 - It provides a complete development environment that handles many aspects of development that makes React behave more like a framework.
 - It provides ES6+ and JSX Support.
 - Consists of a development server with **hot reloading**, which automatically updates the app as you make changes.
 - Faster configuration process.
 - Can contain CSS auto-prefixing
- **React Native**
 - A framework for building mobile applications using JavaScript and React.
 - It allows you to create mobile apps for iOS and Android with a single codebase.
 - It uses native components like **View**, **Text**, and **Button** instead of web components like **div** and **span**.
 - Key feature of React Native is:
 - **Cross-Platform** - can write one codebase that works on both iOS and Android.

➤ React-Vite

- Vite is a fast and modern build tool that aims to provide a better development experience for modern web applications.
- Install vite by using “ npm create vite@latest ”.
- Speeds up the startup time.
- **Native ES Modules in the Browser:** Vite serves your source code as ES modules in the browser, avoiding bundling during development.
- **Hot Module Replacement (HMR):** It allows to update only the part of the code that has changed, without having to reload the whole application.

➔ React Folder Structure

- **public/:** Contains static files that don't change during the build process. The `index.html` file here is the entry point of your React application.
- **src/:** The source code for your React app. Inside src following:
 - **App.js -**
 - The main component of your React app.
 - Contains core structure and content of your app.
 - Build the main layout and combine other components.
 - **index.js -**
 - The entry point of your React application.
 - It sets up the React environment and connects the root React component (usually `App.js`) to the `index.html` file in the public folder.
 - **ReactDOM.render()** method renders the `App` component into the DOM. It looks for an element with the `id="root"` in `public/index.html`.
 - **App.css -**
 - A CSS file for styling the `App` component.
 - Contains styles that apply to the `App` component and its children.

/my-react-app

```
|— /public
  |— index.html      # Main HTML file for rendering the React app
  |— favicon.ico     # Favicon for the app
  |— manifest.json   # PWA (Progressive Web App) manifest
  |— robots.txt      # Web crawlers rules
```

```

├── /src
│   ├── /components      # Reusable components
│   ├── App.js           # Main app component
│   ├── index.js         # Entry point for React DOM rendering
│   ├── App.css          # Global CSS for App component
│   ├── index.css        # Global CSS for the app
│   └── reportWebVitals.js # Optional for measuring performance
├── package.json         # Lists project dependencies and scripts
├── .gitignore           # Specifies files/folders to ignore by Git
├── README.md            # Documentation for the project
└── /node_modules        # Contains all installed npm packages
(auto-generated)

```

→ Components

- These are the building blocks of React.
- Each component defines a part of the user interface (UI).
- Each part of your website can be a component, like a header, a menu, or a footer.
- They are reusable. Once you build a component, we can use it in different places that saves time and effort.

1. Functional Components (import by using `rfce/rafce`)

- Just regular JavaScript functions.
- Accepts some input (called "props") as arguments and returns JSX.
- Used when you just need to display something on the screen.
- They were initially stateless, but with the introduction of **React Hooks**, functional components can now manage state and side effects, making them more powerful.
- **Eg: function Car() {**
 return <h2>Hi, I am a Car!</h2>;
 }

Or

- **function Greeting(props) {**
 return <h1>Hello, {props.name}!</h1>;
 }

Eg (passing props between parent & child components):

Parent Component	Child Component
<pre>import React from 'react'; import Child from './Child'; const Parent = () => { const message = "Hello from Parent"; return (<div> <Child message={message} /> </div>); }; export default Parent;</pre>	<pre>import React from 'react'; const Child = (props) => { return (<div> <p>{props.message}</p> </div>); }; export default Child;</pre>

2. Class Components (import by using rcc)

- More complex.
- These are ES6 classes that extend `React.Component`.
- They can have their own "state", props, and lifecycle methods like `Mounting`, `Updating` and `Unmounting`.
- Used when you need to handle more complex logic, like managing state or reacting to lifecycle events.
- **Eg: class Greeting extends React.Component {**
 render() {
 return <h1>Hello, {this.props.name}!</h1>;
 }}
- Class components must include the “`extends React.Component`” statement.
- It creates an inheritance to `React.Component`, and gives your component access to `React.Component`'s functions.
- Class components have a `render()` method that returns JSX.
- `render()` method is automatically called every time the component's state or props change, causing the component to re-render with updated data.
- `render()` is like a pure function that always returns the output.

Parent Component	Child Component
<pre>import React, { Component } from 'react'; import Child from './Child'; class Parent extends Component { render() { const message = "Hello from Parent"; return (<div> <Child message={message} /> </div>); } } export default Parent;</pre>	<pre>import React, { Component } from 'react'; class Child extends Component { render() { return (<div> <p>{this.props.message}</p> </div>); } } export default Child;</pre>

→ When will components render ?

- **1. Initial Render:**
 - When a component is first mounted to the DOM, it undergoes an initial render.
- **2. State Change:**
 - If the component's **state** changes, React will re-render the component to reflect the updated state.
- **3. Props Change:**
 - If the component receives **new props** from its parent component, it will re-render to update based on the new data passed as props.
- **4. Context Changes:**
 - If the component consumes **context** and the context value changes, the component will re-render with the updated context values.
- **5. Parent Component Re-Renders:**
 - If a parent component re-renders, all of its child components will re-render by default.
- **6. Hooks:**
 - Hooks that update state or trigger side effects can also cause a component to re-render when their state or dependencies change.

- **7. Force Update:**

- Force a component to re-render using the **forceUpdate()** method in class components (though this is rarely recommended as it can lead to performance issues).

→ **Component Composition**

- Component composition includes one or more components in a single component.
- Process of combining multiple smaller components to create more complex, higher-level components.
- It breaks down the UI into manageable pieces, aiding traceability, scalability, and maintenance.
- **Benefits:**
 - Code reusability.
 - Maintainability.
 - In scenarios like lazy loading, composition allows loading only the components needed for the particular time.

→ **What is JSX**

- JSX (JavaScript XML) is a syntax extension for JavaScript that looks similar to HTML.
- It allows developers to write HTML-like code within JavaScript.
- JSX has a tag name, attributes and childrens.
- **JavaScript Inside HTML:** You can embed JavaScript expressions directly within JSX by using curly braces `{ }`.
- **Not HTML:** Even though JSX looks like HTML, it's actually a **React.createElement()** call.
ie,

```
const element = <h1>Hello, world!</h1>;
```

Same as 

```
const element = React.createElement('h1', null, 'Hello, world!');
```

```
const element = React.createElement(  
  'h1', // type of the element (HTML tag)  
  null, // props (no props in this case)  
  'Hello, world!' // children (content inside the tag)  
);
```

- Generally:
 - **First part:** denotes the type of the element. It can be built-in HTML tags like `'div'`, `'h1'`, `'img'`, etc.
 - **Second part:** representing the attributes or properties (called props) for the element. It includes things like `className`, `id`, `src`, `alt`, event handlers like `onClick`, etc. If there are no props, this part can be `null`.
 - **Third part:** denotes the content inside the element. It can be text or anything. For elements that don't have children (like `<img>`), this part can be `null`.
- Some attributes have different names (e.g., `class` => `className` | `for` => `htmlfor` etc).
- If there are multiple elements at the same level, they must be wrapped in a single parent element (like a `div` or `React.Fragment`).

→ Virtual Dom in React

- It is an in-memory representation of the actual DOM.
- A lightweight copy of the actual DOM used to efficiently update the UI.
- Minimizes the number of direct manipulations to the actual DOM.
- When the state changes, first updates the virtual DOM and then compares with the actual DOM (a process known as "**reconciliation**") and updates only the parts of the actual DOM that have changed.
- This makes updates efficient and faster.

HOW IT WORKS ?

1. **Initial Render:** When a component is changed, a virtual DOM tree is created with the added or changed item.
2. **Updating State/Props:** It compares the new virtual DOM with the previous version to see what is changed. This process is called "reconciliation".
3. **Diffing:** Identifies the differences (or "diffs") between both DOM trees.
4. **Updating the Real DOM:** Finally, update only the part of the actual DOM where the new item needs to be added.

→ What is React Element and React Component ?

React Element	React Component
smallest building block of React apps	split the UI into independent, reusable pieces
simple object that describes what you want to see on the screen	is a function or a class that returns a React Element
immutable	mutable
does not have any methods	have their own methods like component mounts, unmounts, updates etc
faster than components because there is no state or anything else	slower than the elements

→ Life Cycle Methods

- ◆ i) Mounting
- ◆ ii) Updating
- ◆ iii) Unmounting
- ◆ iv) Error Handling

● Mounting

- When the component is created and inserted into the DOM.
- ie, components are rendered and made visible in the browser.
- The main lifecycle methods involved in mounting are:

1. **constructor() (optional):**

- Called before the component is mounted.
- It's used to set up initial states or props.

2. **getDerivedStateFromProps() (optional):**

- Called before rendering.
- It's used to update the state based on props.

3. **render():**

- Builds the UI of the component.
- It returns the JSX that represents the component's UI.

4. **componentDidMount():**

- Called after the component is mounted to the DOM.
- Perform operations like fetching data or interacting with the DOM etc.

● **Updating**

- When the component's props or state change.

1. **getDerivedStateFromProps()** (*optional*):

- Updates the state when new props are received.

2. **shouldComponentUpdate()** (*optional*):

- Decides if the component should re-render when there's a change.
- It helps improve performance by preventing unnecessary updates.

3. **render():**

- Rebuilds the UI based on new state or props.

4. **getSnapshotBeforeUpdate()** (*optional*):

- Captures some information just before the update is applied.

5. **componentDidUpdate():**

- Runs after the component has been updated.
- It's used to handle tasks after the component has re-rendered.

● **Unmounting**

- When the component is removed from the DOM.
- The main lifecycle method involved in unmounting is:

1. **componentWillUnmount():**

- This method is called right before the component is removed from the DOM.
- It's used for cleanup activities such as invalidating timers, canceling network requests, or unsubscribing from subscriptions.

● **Error Handling**

- When something goes wrong in the component

1. **componentDidCatch():**

- Runs if there's an error in the component.
- It's used to catch and handle errors, showing a fallback UI if necessary.

DAY 2 - Hooks, State and Props

→ What are hooks ?

- Introduced in React 16.8
- Functions that allow you to use state and other React features in functional components.
- Before Hooks, we can only use these features in class components.
- Essential for managing state, handling side effects, and reusing logic easily, especially in functional components.
- `useState`, `useEffect`, `useContext`, `useReducer`, `useRef`.

★ `useState`:

- used to manage state in functional components.
- used to add a state and the component re-render whenever the state changes.
- takes two values:
 - state - current state.
 - `setState` - function to update the state.
 - `const [state, setState] = useState(initialState)`

Eg: `import React, { useState } from 'react';`

```
function Counter()
{
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

★ `useEffect`:

- to perform side effects, like fetching data or subscribing to events or direct manipulation in DOM.

- takes **two parameters**:
 - **Effect Function**: A function that contains the side effect logic.
 - **Dependency Array** (*optional*): array of dependencies that controls when the effect should run.
 - **No Dependency Array** - runs on **every render**
 - **Empty Dependency Array** - runs **only once** (`componentDidMount`)
 - **With Dependencies** - runs only when **dependencies in the array change** (`componentDidUpdate`)
 - **Multiple Dependencies**: will run when **any one of the dependencies changes**

★ **useReducer**: When you have complex state logic that might involve multiple sub-values or when the next state depends on the previous one.

★ **useRef**:

- used to create a **reference** to a DOM element.
- used to store a mutable value that persists across renders without causing a re-render when it changes.
- this hook is commonly used for directly interacting with DOM elements

★ **useCallback**:

- returns a memoized version of a callback function.
- memoization improves the performance of functions by storing the results of expensive function calls and reusing them rather than recalculating them.
- it helps to optimize the performance of applications by eliminating unnecessary re-renders and unnecessary calculations.
- two parameters:
 - **Callback function**: The function you want to memoize.
 - **Dependencies array**: If these values change, the callback function is re-created.

```
const memoizedCallback = useCallback(
  () => {
    // Your callback logic
  },
  [dependency1, dependency2] // Dependencies array
```

);

★ useContext

- used to pass data across multiple components without passing props through every level of the component tree.
- **How it works:**
 - **Create a Context:** create a context using `createContext()`. It can store the data you want to share across components.
 - **Provider:** wrap your components with the `Provider` component of the created context. It makes the context value available to all components within its subtree.
 - **useContext Hook:** Any component in the subtree can access the context value by using the `useContext` hook.
 - `const value = useContext(MyContext);`

→ State vs Props

State	Props
Components can have their own state. ie, components can change over time (like user input or fetched data).	Short for "properties".
Data is passed within the component only.	Props are used to pass data from one component to another(Parent to Child).
Mutable.	Immutable.
The state is both read and write.	Props are read-only.
The state can be used only with the state class component. But after the introduction of hooks now it can be used in both	Props can be used with state and functional components.

➤ Example for state(with hooks concept and other method):

Eg: <code>import { useState } from 'react';</code>	<code>class Test extends React.Component { constructor() { this.state = {</code>
---	--

<pre>function Counter() { const [count, setCount] = useState(0); return (<div> <p>Count: {count}</p> <button onClick={() =>setCount(count+1)}> Increment</button> </div>); } }</pre>	<pre> id: 1, name: "test" }; } render() { return (<div> <p>{this.state.id}</p> <p>{this.state.name}</p> </div>); } }</pre>
---	--

Example for a props:

Eg : // ParentComponent.js

```
function ParentComponent() {
  const message = "Hello from parent!";
  return <ChildComponent greeting={message} />;
}
```

// ChildComponent.js

```
function ChildComponent(props) {
  return <h1>{props.greeting}</h1>;
}
```

→ Previous state in State

- In React, the "previous state" refers to the value of a component's state before it gets updated.
- When updating state using the `useState` hook, you often need the previous state value to compute the new state.
- `import { useState } from 'react';`
`function Counter() {`

```

const [count, setCount] = useState(0);
const increment = () => {
  setCount((prevCount) => prevCount + 1);
};
return (
  <div>
    <p>Count: {count}</p>
    <button onClick={increment}>Increment</button>
  </div>
);
}

```

- In this example, `prevCount` refers to the previous state value of `count`, ensuring that the new state is calculated based on the most recent state.

→ Conditional Rendering

- Components will display different things depending on different conditions.
- In React, you can conditionally render JSX using JavaScript syntax like if statements, switch statements, && operator, and ternary operators.

- **Using if-else statements**

```

function Greeting({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  } else {
    return <h1>Please sign in.</h1>;
  }
}

```

- **Using Ternary Operator**

```

function Greeting({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? <h1>Welcome back!</h1> : <h1>Please sign in.</h1>}
    </div>
  );
}

```

```
); }
```

- Using Logical AND (&&) Operator

```
function Notification({ hasUnreadMessages }) {  
  return (  
    <div>  
      {hasUnreadMessages && <h2>You have unread messages!</h2>}  
    </div>  
  );  
}
```

- Using switch statement

```
function UserStatus({ status }) {  
  switch (status) {  
    case 'online':  
      return <h2>User is online</h2>;  
    case 'offline':  
      return <h2>User is offline</h2>;  
    default:  
      return <h2>Status unknown</h2>;  
  }}  
}
```

Day 4 (List & Keys, Event Handling, useRef ,useEffect)

→ Lists & Keys

- Lists are used to render a collection of items dynamically.
- In lists we want to map over an array of data and return JSX elements for each item.
- Eg:

```
const items = ['Apple', 'Banana', 'Cherry'];
```

```
import React from 'react';
```

```
function FruitList() {  
  return (  
    <ul>  
      {items.map((item, index) => (  
        <li key={index}>{item}</li>  
      ))}  
    </ul>  
  );  
}  
export default FruitList;
```

→ Keys

- Keys are special string attributes you need to include when creating lists of elements in React.
- Keys should be unique among its siblings but don't need to be globally unique.
- React uses keys to track the items.
- Keys in React helps to identify which items have changed, are added, or are removed.
- Used to minimizing re-renders.
- Without keys, React would have to re-render the entire list every time it updates.
- **Use Unique IDs:** It is better than using the array index, which can lead to issues if the list is reordered or items are inserted/removed.

- **Avoid Using Indexes as Keys:** Using the index of the array as the key can cause issues if the order of items changes, leading to unexpected behavior.

Eg:

```
const items = ['Apple', 'Banana', 'Orange'];

function FruitList() {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>{item}</li> // 'index' is used as a key here
      ))}
    </ul>
  );
}
```

→ Event Handling

- Event handling in React is about responding to user actions like clicks, typing, or submitting forms.
- **Mouse Events:** `onClick`, `onDoubleClick`, `onMouseEnter`, `onMouseLeave`
- **Form Events:** `onChange`, `onSubmit`, `onInput`, `onFocus`, `onBlur`
- **Keyboard Events:** `onKeyDown`, `onKeyPress`, `onKeyUp`
- **Touch Events:** `onTouchStart`, `onTouchMove`, `onTouchEnd`
- React handles events similarly to JavaScript but with some differences.

Key Differences from Traditional DOM Event Handling

1. Event Naming:

- In React, events are named using camelCase instead of lowercase. For example, `onclick` becomes `onClick`.

2. JSX Syntax:

- Instead of a string, you pass a function as the event handler in JSX.

3. Event Binding:

- In React class components, event handlers often need to be bound to the component's context (usually `this`), but in functional components, this is not necessary due to the absence of `this`.

→ Multiple Ways of Fetching Events

- `<button onClick={handleClick}>Click Me</button>`
- `<button onClick={(event)=>handleClick(event, "btn2")}>Click Me</button>`
- `<button onClick={handleClick.bind(this, "btn3")}>Click Me</button>`

→ useRef Hook

- The `useRef` hook in React is used to create a reference to a DOM element or store a value that persists across renders without causing the component to re-render when the value changes.

→ useReducer

- `useReducer` is a React Hook that is used to manage complex state logic in functional components.
- It is an alternative to `useState`.
- But it handles more complicated state logic, that involves multiple sub-values or when the next state depends on the previous one.
- The `useReducer` Hook accepts two main parameters:

`const [state, dispatch] = useReducer(reducer, initialState, init?)`

1. reducer:

- A function that takes the current state and an action, and returns the new state.
- It defines how your state should change with respect to different actions. It takes two arguments:
 - `state`: The current state.
 - `action`: An object that describes what happened.
 - The function returns the new state.

2. initialState:

- The initial state value.

- Starting value of your state.
- It can be any data type, like an object, array, or even a primitive value like a number or string.

3. **init function(optional):**

- A function to lazily initialize the state if the initial state is complex or needs some computation.
- **useReducer** returns an array with two elements:
 - The current **state**.
 - A **dispatch** function that you can use to send actions to the reducer. It is used to trigger state changes by passing an action object.

Eg:

```
import React, { useReducer } from 'react';
Const initialState = {count:0}
```

// Step 1: Define the reducer function

```
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    case 'reset':
      return {count :0 }
    default:
      return state;
  }}

```

```
function Counter() {
```

// Step 2: Use useReducer hook

```
const [state, dispatch] = useReducer(reducer,initialState);
```

```

return (
  <div>
    <p>Count: {state.count}</p>

    {/* Step 3: Dispatch actions */}
    <button onClick={() => dispatch({ type: 'increment' })}>+</button>
    <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
    <button onClick={() => dispatch({ type: 'reset' })}></button>
  </div>
);
}

export default Counter;

```

When to use <code>useState</code>	When to use <code>useReducer</code>
If the state is simple, such as a single value or a small set of independent variables.	If the state is more complex, such as an object or array contains multiple sub-values
When the state transitions (changes) are straightforward.	When the next state depends on the previous state.
The state changes don't require complex logic or calculations.	The state changes involve more complex logic (e.g., multiple conditions, complex calculations).

- We can relate `useReduce` with Javascript `Reduce`:

JAVASCRIPT REDUCE	REACT USEREDUCER
REDUCE(REDUCER, INITIALVALUE)	USEREDUCER(REDUCER, INITIALVALUE)
REDUCER(ACCUMULATOR, CURRENTVALUE)	REDUCER(STATE, ACTION)
REDUCER RETURN A VALUE	REDUCER RETURN NEW STATE
REDUCE METHOD RETURN SINGLE VALUE	*USEREDUCER RETURN PAIR OF VALUES NEWSTATE & DISPATCH

→ JSON-Server

- It is a simple tool that allows you to create a full fake REST API with zero coding.
- Creating a node + express + mongodb backend is a time consuming process.
- So it provides a quick and easy way to simulate a backend server.
- It quickly set up a fake REST API.
- It's commonly used for prototyping, mocking APIs, or serving as a simple backend for a frontend application.

→ To Setting Up a JSON Server

- Install JSON Server using npm (Node Package Manager).
npm install -g json-server
- Create **db.json** file
- Start json server by - **json-server --watch db.json**
- We can change port number by - **json-server --watch db.json--port 4000**
- Now we can access the data using - **http://localhost:4000**

→ Interacting with the API

- Can perform CRUD (Create, Read, Update, Delete) operations using HTTP methods like GET, POST, PUT, PATCH, and DELETE.
- **GET**: Retrieves data from the server.
- **POST**: Adds new data to the server.
- **PUT**: Updates existing data on the server by replacing it.
- **PATCH**: Updates part of the existing data.
- **DELETE**: Removes data from the server.
- GET <http://localhost:4000/products> will return all products.
- GET <http://localhost:4000/products/1> will return the products with `id=1`.
- **Filter**
 - GET <http://localhost:4000/products?category=electronics>
 - If more than one criteria we can use '&'
 - [products?category=electronics&discount.type=shipping](http://localhost:4000/products?category=electronics&discount.type=shipping)
- **Sort**
 - GET http://localhost:5000/products?_sort=price&_order=asc
 - GET http://localhost:5000/products?_sort=price&_order=desc
- **Relationship**
 - Handling relationships between data can be done using either **embedding** or **expanding** techniques.
 - **Embedding** involves including related data directly within the main resource.
 - Makes it easy to access related information in a single request.
 - **Expanding** involves separating related data into different resources and linking them via identifiers (like IDs).
 - This method is useful for reducing data duplication
 - GET http://localhost:5000/posts?_expand=category
- **Pagination**
 - Used to divide a large set of data into smaller, more manageable chunks or "pages."
 - Instead of loading all the data at once, pagination allows users to view the data page by page.
 - `json-server` automatically supports pagination using two parameters. They are `_page` & `_limit`.

- `_page`: Specifies the page number.
- `_limit`: Specifies the number of items per page.
- GET `http://localhost:3000/posts?_page=1&_limit=5`
- **Operators**
 - **Less Than or Equal To (`lte`):**
 - `http://localhost:3000/products?price_lte=25`
 - **Greater Than or Equal To (`gte`):**
 - `http://localhost:3000/products?price_gte=30`
 - **By combining:**
 - `http://localhost:3000/products?price_gte=20&price_lte=30`
- **Searching**
 - We can search a text using `q` as a query parameter.
 - GET `http://localhost:3000/products?q=ing`

→ Axios

- It is a popular JavaScript library used for making HTTP requests from a web browser.
- Axios allows you to send GET, POST, PUT, DELETE requests.
- It manages the request and response data, and handling errors efficiently.
- First, we need to install Axios by using - `npm install axios`.
- Axios supports the Promise API.
- `axios.get()`: used to make a GET request by passing the URL of the API we want to fetch.
 - `axios.get('https://api.example.com/data')
 .then(response => console.log(response.data))
 .catch(error => console.error(error));`
- `axios.post()`: used to send data to the server.
 - `axios.post('https://api.example.com/data', { key1: 'value1',key2: 'value2'})
 .then(response => console.log(response.data))
 .catch(error => console.error(error));`
- `axios.put()`: used to update existing data on a server.

- `axios.put('https://api.example.com/data/1', {key1: 'newValue'})`
`.then(response => console.log(response.data))`
`.catch(error => console.error(error));`
- **axios.delete()**: used to delete data from a server.
 - `axios.delete('https://api.example.com/data/1')`
`.then(response => console.log(response.data))`
`.catch(error => console.error(error));`
- When you send a request to the server, you will receive a response object from the server with the properties given below.
 - **data**: The response body, which contains the actual data sent from the server.
 - **status**: The HTTP status code from the server response (e.g., 200).
 - **statusText**: HTTP status message returned by the server.
 - **headers**: An object containing all the response headers.
 - **config**: The configuration object that was used to make the request..
 - **request**: actual XMLHttpRequest object.

- You will get an error object if there will be a problem with the request. It contains:
 - **message**: Error message text.
 - **response**: Response object (if received).
 - **code**: A code representing the error.
 - **request**: Actual XMLHttpRequest object (when running in a browser).

→ useContext

- **useContext** is used to share or pass values between multiple levels of components.
- It provides a way to pass data through the component tree without having to pass props.
- It's useful for sharing data like themes, user authentication, or settings across multiple components without prop drilling.

→ Why do we use useContext?

- **Prop Drilling:** In a React app, data is often passed from parent to child components via props. However, when you need to pass data through multiple nested components, this can lead to "prop drilling".
- **Nested Providers:** You can nest multiple providers to manage different pieces of context. Each provider can have its own value.
- Steps to use:
 - Create a Context
 - `createContext()`

```
import React, { createContext, useContext } from 'react';
const MyContext = createContext();
```
 - Provide the Context Value
 - Wrap the components in a `MyContext.Provider` that need access to the context.
 - Pass the value - `value{ }`

→ Higher Order Component (HOC)

- Used for reusing component logic.
- An HOC is a function that takes a component as an argument and returns a new component with additional functionality.
- **HOCs do not modify the original component**, but wrap it with another component that provides extra features.
- HOCs reuse logic across multiple components without repeating code.
- HOCs are helpful for adding cross-cutting concerns (like loading, authentication) without duplicating code.
- **Cross-cutting** concerns refers to functionalities that are used at multiple parts of an application. (For example, if you have multiple components that need the same authentication logic, instead of repeating that logic, you can create an HOC.)
- HOC works by transforming props passed to the wrapped component.
 - `const enhancedComponent = highOrderComponent(OriginalComponent`

Eg: If we use the same counting logic in two or more components we can share the logic.

Counter 1	Counter 2	HOC
import React from 'react'	import React from 'react'	import React, {useState} from

<pre>import Update from './HOC'; const Counter1 = ({counting, increase}) => { return (<div> <h1>This is Counter-1</h1> <button onClick={increase}>Clicked me {counting} Times</button> </div>) } export default Update(Counter1)</pre>	<pre>import Update from './HOC'; const Counter2 = ({counting, increase}) => { return (<div> <h1>This is Counter-2</h1> <button onClick={increase}>Clicked me {counting} Times</button> </div>) } export default Update(Counter2)</pre>	<pre>'react'; const Update = (OriginalComponent) => { function NewComponent(){ const [count,setCount] = useState(0); const Up = () => { setCount(count + 1); }; return (<OriginalComponent counting={count} increase={Up} />) } return NewComponent; } export default Update</pre>
---	---	---

→ React Router Dom

- React Router DOM is a widely used and fully featured library for handling client-side routing in React applications .
- It enables developers to create single-page applications (SPAs) with multiple pages without requiring full page reloads.
- That is, when you navigate between pages it will only refresh the content inside your Routes component. All the rest of the content on your page will stay the same.
- It helps to enhance the performance and user experience.
- It provides unique URLs for different components in the app and makes it easily shareable.

Key Features

1. **Declarative Routing:** Define routes using JSX, making the routing logic clear and easy to manage.
2. **Nested Routes:** Organize routes hierarchically by routes within other routes to create complex layouts.
3. **Redirects and Navigation:** Use `<Navigate>` for redirects and programmatic navigation with hooks like `useNavigate`.

4. **Dynamic Routing with URL Parameters:** Handle dynamic segments in URLs using route parameters by using hooks like `useParams`.
5. **Code Splitting:** Integrate with React's lazy loading to split code and load components on demand, enhancing performance.
6. **Components:** Utilize components like `<Link>`, `<Route>`, `<Routes>`, etc., for versatile routing capabilities.
7. **Protected Routes:** Restrict access to certain routes based on authentication or other conditions by creating protected routes.
8. **Browser and Hash Routing:** Support different routing strategies using `<BrowserRouter>` or `<HashRouter>`.

→ Hooks Provided by React Router DOM

- **useParams:** Access the dynamic parameters from the current URL, such as an ID.
- **useNavigate:** Programmatically navigate between routes.
- **useLocation:** Access the current location object.
- **useMatch:** Match the current URL against a pattern, useful for conditional rendering based on the route.
- **useSearchParams:** Used for work with query parameters.
- **useOutlet:** Access the child route elements to be rendered in the parent route component.
- **useOutletContext:** Access the context passed from the parent route using the `<Outlet>` component.
- **useRoutes:** Used as an alternative way to using the `<Routes>` component. Used to provide multiple routes like array of objects.

Eg:

```
const routes = [  
  { path: '/', element: <Home /> },  
  { path: '/about', element: <About /> }  
];  
  
const App = () => {  
  const element = useRoutes(routes);  
  return element;  
};
```

→ **useHref**: Returns a URL that can be used to link to a given location.

→ Common Components

- **BrowserRouter**: Used to keep the UI in sync with the URL.
- **HashRouter**: Uses the hash portion of the URL (e.g., [example.com/#/about](#)) to keep the UI in sync.
- **Routes**: A container for a list of `<Route>` elements.
- **Route**: Defines a mapping between a URL path and a component.
- **Link**: Renders an anchor tag for navigation without full page reloads.
- **NavLink**: Similar to `<Link>`, but allows applying active styles when the link is active.
- **Outlet**: Placeholder for rendering child routes in nested routing.

→ To Setting Up a Router

- Install React Router DOM using npm or yarn:
 - **npm install react-router-dom** OR **yarn add react-router-dom**
- Setting Up the Router:
 - **Wrap your application with the `<BrowserRouter>` component.**
- Create the components:
 - **Create the components that need to be included in the App.**
- Defining Routes:
 - **Use the `<Routes>` and `<Route>` components to define your application's routing structure.**
 - **Define a single `<Route>` component for each route in your application and then put all those `<Route>` components in a single `<Routes>` component.**
 - **Each `<Route>` accepts a path that matches the url.**
- For Links:
 - **Use the `<Link>` tag to render as an anchor tag for navigation.**
 - **Inside the `<Link>` we use 4 props.**
 - **to**: it is used to set the URL. (`<Link to="/">Home</Link>`).
 - **replace**: it is a boolean that when set to true it will replace the current page in the browser history.

- **reloadDocument:** it is also a boolean, if it is set to true it will do a full page refresh instead of just re-rendering the content inside your Routes component.
- **state:** pass data along with your **Link** that does not show anywhere in the URL.
- **<NavLink>** also used to create navigational links but have different features.
- **<NavLink>** contains an 'active' class by default. So we can highlight the currently active link.
- For Navigating Programmatically:
 - Using **useNavigate** can perform redirects and programmatic navigation.
- If No Match Route(page not found):
 - By adding ***** in the path.
 - This route should be placed at the end of your **<Routes>** to ensure it only catches unmatched routes. (`<Route path="*" element={<NotFound />} />`)